ParadigmForge

A Governed Architecture for Machine-Generated Mathematical Theory Formation

Maximiliano Lucius
Independent Researcher / Aureus Technologies
maximiliano@aureus-finance.com

Draft — July 2026

Abstract

Autonomous and semi-autonomous systems increasingly produce plausible research-level mathematics, and a growing body of work governs when a *generated statement* may become trusted knowledge. Proving statements inside a fixed formal language, however, is not the whole of mathematical progress. Frontier mathematics has repeatedly advanced by *changing the representational and conceptual space* in which problems are posed—by inventing techniques, building bridges between domains, and introducing new definitions, invariants, and objects that expose previously hidden structure. We present ParadigmForge, a *governed theory-formation architecture* that extends the Bourbaki Engine research program from claim promotion to the generation, testing, refinement, governance, and validation of candidate mathematical techniques, cross-domain bridges, conceptual languages, and candidate domains. ParadigmForge organises discovery into four escalating levels of novelty (novel technique; cross-domain bridge; new conceptual language; domain genesis) and enforces a single governing principle: *high epistemic temperature for exploration, near-zero temperature for promotion.* A high-temperature speculative zone—Frontier Atlas, TechniqueForge, BridgeForge, ConceptForge, Experimentarium, and DomainFoundry—proposes radical candidates but holds *no* authority to promote them; a near-zero-temperature trust zone (the Mirador representation, formalization, and CongressBench adversarial review) alone may admit a result into trusted knowledge, and every rejected proposal is retained as structured negative knowledge. We contribute (i) an operational taxonomy of mathematical novelty; (ii) a typed epistemic representation for techniques, bridges, concepts, failures, and candidate domains, specified as an extension of Mirador; (iii) a discovery-to-promotion workflow that separates speculative exploration from trusted knowledge; (iv) a multidimensional novelty model with explicit anti-gaming controls that does not reduce novelty to textual or embedding distance; and (v) ParadigmForgeBench, a four-track evaluation framework for technique-, bridge-, concept-, and domain-level novelty. We are explicit about maturity: ParadigmForge is an architecture and evaluation-framework proposal built on the separately published Mirador and ProofContext prototypes; **no ParadigmForge experiment has been run and no empirical result is claimed**. Every results table is a labelled placeholder, and the complete execution plan is preregistered separately. The paper's argument is deliberately conservative: mathematical AI will not reach frontier discovery merely by proving more statements inside fixed languages; it must also learn to propose, test, reject, and govern changes to the conceptual language of mathematics itself.

Contents

1 Introduction

A model that emits an unfamiliar-looking theorem has not, by that act, made a discovery. Whether a statement constitutes progress depends on properties that fluency does not establish: it must be true (or its falsity must be a useful counterexample), it must be novel, nontrivial, and useful to subsequent work, and—at the frontier—it often must be expressible at all, which is not guaranteed by the language currently available. The last point is the concern of this paper. Much recent progress in AI for mathematics has been organised around a fixed target language: given a statement in Lean or in natural language, produce a derivation a checker accepts (Polu and Sutskever, 2020; Yang et al., 2023; Trinh et al., 2024). This is the right frame for olympiad-style problems and for filling a known gap in a formal library, and it has produced striking results. But research mathematics is not only the discharge of statements in a fixed language. Its most consequential moves are frequently changes to the language itself: a new technique that makes a previously intractable estimate routine; a bridge that transports a theorem from one domain to another; a definition or invariant that compresses several disconnected phenomena into one; and, occasionally, the emergence of an entire productive domain around such a language.

The Bourbaki Engine program (Lucius and Minich, 2026a) has developed an infrastructure for the *governance* of machine-assisted mathematics: a typed, versioned representation whose unit is a Mathematical Cell (Lucius, 2026c); a dependency-aware retriever that returns the minimal verifiable context in which a claim can be used safely (Lucius, 2026d); a propose-only discovery plane of operators (Lucius and Minich, 2026c); a benchmark for the reliability of adversarial review (Lucius and Minich, 2026b); and a benchmark for faithful ingestion of the literature (Lucius, 2026b). The spine of that program is the separation of a Discovery Plane, which may be speculative and internally contradictory, from a Trust Plane, which alone may promote a claim into load-bearing knowledge. That program governs *accumulation*. It does not, on its own, address the production of new representational and conceptual structure. A machine that only prevents false promotions can sit forever at an empty frontier.

Contribution of this paper. ParadigmForge extends the Bourbaki Engine from governed claim promotion to *governed theory formation*. It is a layer above TheoryForge that targets the creation, testing, and adversarial validation of *techniques*, *cross-domain bridges*, *conceptual languages*, and *candidate domains*—not merely conjectures and reductions inside an existing theory. Its central design commitment is a governed separation of temperatures: a high-temperature speculative zone proposes radical candidates without any promotion authority, and a near-zero-temperature trust zone admits results only through formalization and fresh-context adversarial review, retaining every rejection as structured negative knowledge (Fig. 1).

Maturity, stated plainly. This is an *architecture and evaluation-framework* paper. The substrates it depends on (Mirador, ProofContext) are separately published prototypes; the ParadigmForge layer itself is specified, not built. **No ParadigmForge experiment has been run, and this**

paper reports no accuracy, success rate, expert assessment, or mathematical discovery. Results tables appear only as clearly marked placeholders; the executable experimental program is preregistered in the accompanying roadmap. We adopt the companion program's evidential discipline throughout: components are labelled *implemented*, *specified*, or *planned*, and design goals are never written as if measured.

Research questions. The paper is organised around six questions, none of which it answers empirically; it specifies how each becomes answerable.

RQ1
Can a governed AI system reconstruct mathematically productive *techniques* from pre-discovery knowledge, without collapsing into memorised rediscovery?

RQ2
Can typed structural alignment produce *theorem-transporting* bridges that outperform semantic analogy?

RQ3
Can an AI-generated *concept* demonstrate explanatory compression and predictive utility on held-out mathematical cases?

RQ4
Which governance mechanisms most effectively prevent cosmetic novelty and false theory promotion?

RQ5
What evidence is sufficient to distinguish a *productive mathematical domain* from an overfitted collection of definitions?

RQ6
Does structured *negative knowledge* improve mathematical exploration by preventing repeated failure?

Claims. We claim, as architectural and methodological contributions only: (1) a governed architecture for machine-assisted mathematical theory formation (§6); (2) an operational taxonomy of mathematical novelty (§6, §8); (3) a typed representation for techniques, bridges, concepts, failures, and candidate domains (§7); (4) a discovery-to-promotion workflow separating speculative exploration from trusted knowledge (§5, §12); (5) ParadigmForgeBench, an evaluation framework for technique-, bridge-, concept-, and domain-level novelty (§9); (6) a research methodology for testing mathematical productivity without conflating it with textual novelty (§10); and (7) an extension of the Bourbaki Engine toward controlled conceptual evolution (§4). No claim in this paper depends on an experiment we have not run.

2 From Theorem Proving to Theory Formation

We fix a set of distinctions that the rest of the paper depends on. Each names a boundary that is easy to blur and expensive to blur.

Theorem proving vs. theory formation. Proving derives a fixed statement inside a fixed language. Theory formation changes what statements are available to prove: it introduces definitions, invariants, techniques, and translations, and only then proves within the enlarged language.

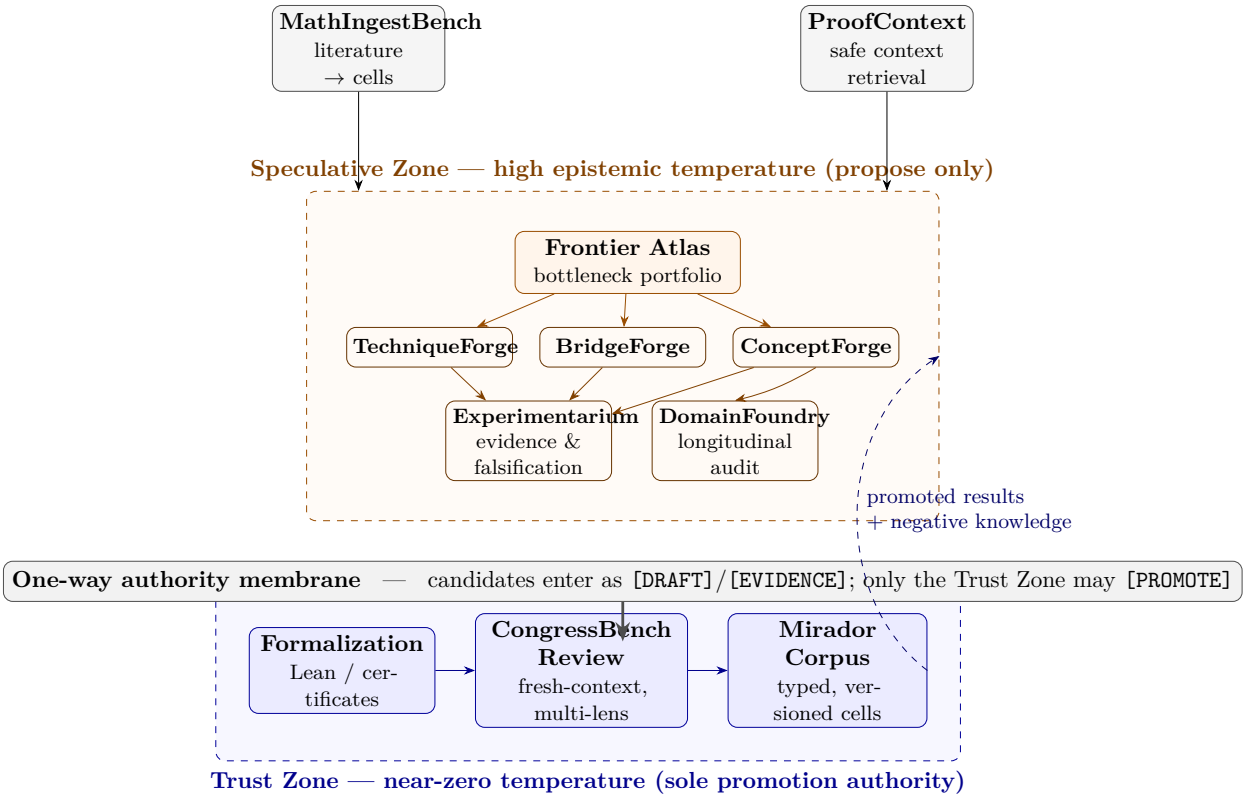

Figure 1: The ParadigmForge architecture. A high-temperature *speculative zone* (orange) proposes candidates but holds no promotion authority; a near-zero-temperature *trust zone* (blue) alone admits results into the Mirador corpus, through formalization and CongressBench review. The two are joined by a one-way authority membrane. MathIngestBench feeds the corpus the Frontier Atlas reads; ProofContext supplies safe context; promoted results *and* negative knowledge flow back to the Atlas. Every component has an evaluation mechanism (§9).

The former has a well-posed success predicate (a checker accepts); the latter does not, which is precisely why it needs governance rather than a single score.

Conjecture generation vs. conceptual invention. A conjecture is a statement in the existing language whose truth is open. A concept is a new piece of language—an object, an equivalence, an invariant—whose value is not truth but productivity. A system can generate endless conjectures without inventing a single concept.

Analogy vs. a valid mathematical bridge. An analogy asserts resemblance. A bridge is a typed, partial, structure-preserving map that *transports* at least one nontrivial theorem and states where it breaks. Analogy is cheap; transport is the evidence.

Renaming vs. genuine abstraction. Renaming preserves content-hash identity (Lucius, 2026c); abstraction changes which theorems are provable or which proofs are shorter. The first is detectable mechanically; the second must be demonstrated.

Local novelty vs. productive novelty. A construction can be new-to-the-corpus yet inert. Productive novelty yields theorems, transfers across problems, and is used by agents other than its proposer.

Empirical evidence vs. proof. Exact enumeration, simulation, and pattern-finding generate evidence and falsification pressure; they do not establish theorems. A numeric grid is never a proof (Lucius and Minich, 2026a).

Discovery vs. epistemic promotion. Producing a candidate is discovery in the weak sense; admitting it as trusted knowledge is promotion, a governed act with a different authority.

Open-ended exploration vs. trusted knowledge. The two require opposite virtues—diversity and permissiveness on one side, conservatism and auditability on the other—and must not share authority.

These distinctions are not rhetorical. Each corresponds to a concrete failure mode that ParadigmForge is built to detect: renaming as novelty, analogy without transport, evidence promoted as proof, a concept fitted only to the examples that motivated it. §8 makes each measurable.

The literature of *automated theory formation* is older than the current wave of language models. Lenat's AM explored concepts and conjectures by heuristic search (Lenat, 1976); Colton's HR systematised concept invention and the problem of "interestingness" (Colton, 2002). Lakatos gave the philosophical account that this paper operationalises: mathematical knowledge grows not by monotone accumulation but by the interplay of conjecture, proof, and refutation, in which counterexamples reshape definitions themselves (Lakatos, 1976). ParadigmForge can be read as a governed, machine-scale realisation of that dialectic, with the crucial addition—absent from the classical systems—of an explicit separation between proposing and promoting.

3 Background and Related Work

We situate ParadigmForge against seven strands of work, citing primary sources and marking, for each, the boundary ParadigmForge adds. We claim novelty only in the composition, not in any single mechanism.

Automated and formal theorem proving. The mechanisation of proof runs from the Logic Theory Machine (Newell et al., 1957) and proof planning (Bundy, 1988) to McCune's resolution of the Robbins conjecture (McCune, 1997), and to the large computer-assisted and formal proofs that fixed the modern standard of certified computation: the Four Colour Theorem (Gonthier, 2008), the Kepler conjecture (Hales et al., 2017), and the Boolean Pythagorean triples problem (Heule et al., 2016). Interactive proof assistants—Lean 4 and its library (de Moura and Ullrich, 2021; The mathlib Community, 2020)—supply the trusted kernels ParadigmForge treats as one verification backend among several. This lineage certifies proofs; it does not generate techniques or concepts.

Neural and LLM theorem proving. Learned provers—GPT-f (Polu and Sutskever, 2020), HyperTree Proof Search (Lample et al., 2022), retrieval-augmented proving with LeanDojo (Yang et al., 2023), and the DeepSeek-Prover line (Xin et al., 2024a,b)—search for derivations of *given* statements, and autoformalization aligns informal statements to formal ones (Wu et al., 2022; Jiang et al., 2023). Benchmarks such as miniF2F (Zheng et al., 2022), the MATH dataset (Hendrycks et al., 2021), GSM8K (Cobbe et al., 2021), Minerva (Lewkowycz et al., 2022), and FrontierMath (Glazer et al., 2024) measure proving or problem-solving on fixed statements, and research-level benchmarks such as First Proof (Abouzaid et al., 2026) raise the difficulty to unpublished problems—but all still score derivation of a *given* statement. ParadigmForge consumes such provers as components but is evaluated on a different axis: the productivity of newly proposed language.

AI-assisted discovery. AlphaGeometry and its successor solve olympiad geometry by coupling a language model to a symbolic engine with auxiliary-construction search (Trinh et al., 2024; Chervonyi et al., 2025); AlphaProof reached competition-level formal proving (Google DeepMind, 2024). Program-search discovery—FunSearch (Romera-Paredes et al., 2024) and AlphaEvolve (Novikov et al., 2025)—optimises an executable against a fixed evaluator and has produced genuinely new constructions (e.g. improved cap-set and packing bounds). These systems change *artifacts* against a fixed objective; ParadigmForge targets change to the *objective language* itself, under governance, and measures it by transfer and productivity rather than by a single evaluator score. Davies et al. showed AI can guide human mathematical intuition toward new theorems (Davies et al., 2021); ParadigmForge aims to make the intuition-shaping step itself an auditable, governed artifact.

Open-ended search and quality–diversity. The exploration side of ParadigmForge inherits from novelty search (Lehman and Stanley, 2011) and quality–diversity methods such as MAP-Elites (Mouret and Clune, 2015; Cully et al., 2015), which maintain diverse populations rather than optimising a scalar. ParadigmForge's speculative zone is a quality–diversity population over *typed mathematical objects*, with the essential addition that diversity is not enough: a candidate must survive transport, falsification, and adversarial review before it counts.

Analogy and structural transfer. Structure-mapping theory formalises analogy as alignment of relational structure rather than surface features (Gentner, 1983). BridgeForge (§6.3) is a mathematical, transport-obligated realisation of that idea: alignment is necessary but insufficient; the bridge must carry a theorem and declare its breakage point.

Mathematical knowledge representation, provenance, and truth maintenance. Truth-maintenance systems introduced epistemic states and dependency-directed invalidation (Doyle, 1979; de Kleer, 1986); provenance and transparent-log methods (Kuhn et al., 2016; Moreau and Missier, 2013; Laurie et al., 2013) and build-level invalidation (Çelik et al., 2017) inform the Mirador substrate. ParadigmForge extends that substrate with node and relation types for theory formation (§7).

Philosophy and practice of mathematical discovery. Beyond Lakatos (Lakatos, 1976) and Pólya (Pólya, 1945), the paper takes seriously the measure-of-intelligence view that genuine novelty is skill-acquisition on unseen structure, not recall (Chollet, 2019); this motivates the contamination controls and held-out designs of §9.

4 Bourbaki Engine Context

ParadigmForge is not an isolated system; it is an extension of an existing, partly implemented program. We summarise the substrate so the paper is self-contained.

Mirador (Lucius, 2026c) represents mathematical knowledge as a typed, versioned *Mathematical Cell*: a canonical claim bound to its context (assumptions, exclusions, regularity, scope), typed dependency edges (each carrying its own soundness obligation), artifacts, provenance, and an *epistemic state*. Two commitments are load-bearing for ParadigmForge: object *type* is orthogonal to epistemic *state* (being a "theorem" does not make a cell `[PROMOTED]`); and claim identity is the canonical hash of statement-plus-context, so a mere renaming collides with its source and cannot

Table 1: Capability comparison with representative systems. • = provided; ○ = partial or scoped; – = not provided. Columns: *Prove* a fixed statement; generate *Conj*ectures; synthesise *Conc*epts / definitions; construct cross-domain *Bridge*s with theorem transport; *Gov*erned promotion (separation of powers); retain structured *Neg*ative knowledge. Cells for ParadigmForge denote capabilities the architecture *specifies*; the layer is not yet implemented (§13).

System	Prove	Conj	Conc	Bridge	Gov	Neg
Lean / mathlib (de Moura and Ullrich, 2021; The mathlib Community, 2020)	•	–	–	–	○	–
AlphaGeometry(2) (Trinh et al., 2024; Chervonyi et al., 2025)	•	○	–	–	–	–
DeepSeek-Prover (Xin et al., 2024b)	•	–	–	–	–	–
FunSearch / AlphaEvolve (Romera-Paredes et al., 2024; Novikov et al., 2025)	○	○	–	–	○	–
TheoryForge (Lucius and Minich, 2026c)	○	•	○	○	•	○
ParadigmForge (this paper)	○	•	•	•	•	•

be sold as new. Invalidation propagates deterministically over an append-only, hash-chained event log.

ProofContext (Lucius, 2026d) compiles a query into a typed Problem Signature and returns the smallest *Evidence Bundle* in which a claim can be used safely, rejecting near-matches that are incompatible on a typed axis regardless of similarity, and holding no authority to promote.

TheoryForge (Lucius and Minich, 2026c) is the propose-only Discovery Plane: a library of operators emitting typed candidate cells at epistemic state at most `[EVIDENCE]`.

CongressBench (Lucius and Minich, 2026b) measures the *False Theorem Promotion Rate* (FTPR) of adversarial, fresh-context review across a ladder of reviewer independence.

MathIngestBench (Lucius, 2026b) measures faithful ingestion of the literature into cells—the front of the pipeline that populates the corpus the Frontier Atlas reads.

Bourbaki Engine (Lucius and Minich, 2026a) composes these into a two-plane architecture (Discovery vs. Trust) with a one-way authority membrane, certificate discipline (search ≠ evidence ≠ certificate ≠ checker), and a readiness ladder $R0$–$R9$ whose top rungs the engine cannot self-declare.

ParadigmForge occupies a specific place in that ladder. The engine's ladder reaches, at $R3$–$R4$, the regeneration of nontrivial known results and the production of a genuinely new lemma, counterexample, reduction, or barrier on a bounded open problem. ParadigmForge is the machinery aimed at that transition and beyond: where TheoryForge proposes conjectures and reductions *inside* a language, ParadigmForge proposes and governs changes *to* the language—techniques, bridges, concepts, and, over long horizons, domains. It reuses Mirador for representation, ProofContext for retrieval, and a CongressBench-compatible reviewer for review; it adds the six components of §6, the typed extensions of §7, and the evaluation framework of §9.

5 Design Principles

The architecture follows from a small number of principles. Each is stated so that a design that violates it can be identified.

Principle 1 (Representational change is a first-class goal)**.** The system's objective is not only to prove statements in a fixed language but to propose, test, and govern changes to that language:

new techniques, bridges, concepts, and—emergently—domains. Success is measured by productivity and transfer, not only by proof accuracy.

Principle 2 (Temperature separation)**.** *High epistemic temperature for exploration; near-zero epistemic temperature for promotion.* Speculative agents may propose radical, imperfect, even internally contradictory ideas; they must never promote their own results into trusted knowledge. Promotion is a distinct, conservative act performed by an independent authority.

Principle 3 (Transport over resemblance)**.** A proposed bridge or analogy counts only if it transports a nontrivial mathematical statement and states where it fails. Semantic similarity proposes; a checkable transport adjudicates.

Principle 4 (Necessity over decoration)**.** A proposed technique or concept counts only if an ablation shows its new component is *counterfactually necessary*—that removing it fails to close the target obligation or loses the theorem yield. Aesthetic reformulation without necessity is rejected.

Principle 5 (Productivity, not proclamation)**.** A "new domain" cannot be declared by an agent. It is recognised only through sustained productivity: a family of objects, internal operations and invariants, multiple theorems across more than one problem family, natural internal questions, transfer to established domains, and use by agents other than the proposer.

Principle 6 (Failure is knowledge)**.** Rejected proposals are not discarded. Each is retained as structured negative knowledge—a failed approach, a counterexample, an obstruction or barrier certificate, with an explanation of *why* it fails—and reused to prune future exploration.

Principle 7 (Novelty is multidimensional)**.** Mathematical novelty is not textual, embedding, or retrieval distance. It is a vector of structural change, operational change, explanatory compression, theorem productivity, transfer breadth, counterfactual necessity, robustness, independent rediscovery, and verification coverage. No scalar fully determines it.

Principles 2 and 6 are the governance core; 3 and 4 are the anti-gaming core; 5 is the criterion that keeps "domain" honest.

6 ParadigmForge Architecture

ParadigmForge comprises six components in the speculative zone and a governance path into the trust zone (Fig. 1). We specify each by its input, its output (always typed Mirador cells at epistemic state at most `[EVIDENCE]`), its operators, and—critically—its *acceptance gate*, the check a candidate must survive to leave the component. Every component has a corresponding evaluation mechanism in §9.

The four levels of novelty the architecture is designed to reach are shown in Fig. 2. They are escalating in representational change and in difficulty of validation; a system may operate productively at $L1$ for a long time before any $L3$ candidate survives.

6.1 Frontier Atlas

The Frontier Atlas is a structured representation of the region around a mathematical problem. Its purpose is to convert a broad objective—"resolve the Birch–Swinnerton-Dyer conjecture"—into a *portfolio of precise research bottlenecks*, each of which is an investigable target rather than a slogan.

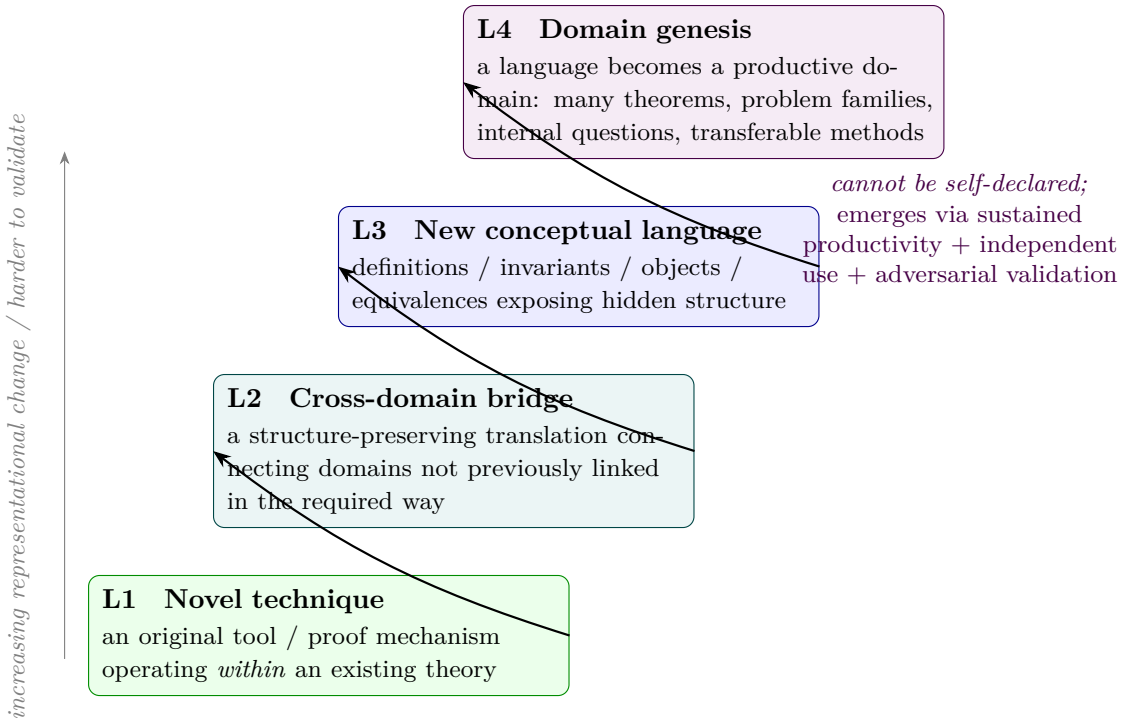

Figure 2: The four escalating levels of mathematical novelty ParadigmForge targets. Higher levels change more of the representational space and are correspondingly harder to validate; $L4$ cannot be self-declared and requires longitudinal assessment (§6.6, §9).

Content. For a target problem the Atlas encodes, as typed Mirador cells and edges: known theorems and partial results; proof techniques and the regimes in which they work; unresolved subproblems; conceptual gaps; failed approaches; known barriers and no-go results; counterexamples and limiting cases; unexplained computational patterns; assumptions that recur across attempts; and, most importantly, the *precise points at which existing methods stop working*. The last category is the Atlas's distinctive output: a bottleneck is a cell of type ResearchBottleneck that names the exact obligation no current technique discharges, together with the techniques that approach it and the point at which each fails.

Operators and gate. The Atlas is built by ingestion (MathIngestBench) and retrieval (ProofContext), then refined by *decomposition* operators that split a target into subproblems and localise failure. A bottleneck leaves the Atlas only if it is (i) grounded in cited results, (ii) specific enough to admit a candidate technique, bridge, or concept, and (iii) not already discharged by a known method in the corpus. The negative-knowledge store feeds the Atlas directly: a family of approaches that fail for a common reason becomes a single, high-value cell.

6.2 TechniqueForge

TechniqueForge searches for novel *techniques*—tools or proof mechanisms operating primarily inside or near an existing theory ($L1$). It consumes a bottleneck and emits candidate techniques as typed cells with a transport/soundness obligation.

Operators. Generative operators include: *composition* of two partially successful techniques; *dualization*; *localization* and *globalization*; *deformation* (introducing a parameter and studying the limit); *relaxation* of a hypothesis; *change of scale*; *randomization* of a deterministic construction; *discretization* and *continuous-limit* passage; *quantitative strengthening* of a qualitative argument; *transfer of a proof schema*; and *extraction of an invariant from a successful algorithm.* These generalise the operator library of TheoryForge (Lucius and Minich, 2026c) to the level of technique construction.

Acceptance gate (necessity ablation). By Principle 4, a candidate technique is accepted for downstream review only if an *ablation* demonstrates that its novel component is mathematically necessary: removing or replacing that component with the nearest existing tool must fail to close the target obligation, or must lose a theorem the full technique yields. A technique that merely reformulates an existing argument, however elegantly, does not pass. All numeric witnesses used in the ablation are re-checked in exact arithmetic (Lucius and Minich, 2026a).

6.3 BridgeForge

BridgeForge searches for structure-preserving *bridges* between domains ($L2$). It is the component most sharply distinguished from analogy detection: a bridge is not a claim of resemblance but a typed, partial map that carries mathematics across.

What a bridge aligns. A candidate bridge aligns mathematical objects, operations, relations, symmetries, invariants, proof patterns, and obstruction structures between a source and a target domain. The alignment is *typed*: each correspondence records which structure it preserves and under what hypotheses. Bridges are treated as *partially functorial*—they need not be total or invertible—and *bidirectional* where possible, so that transport can be attempted in both directions.

Acceptance gate (transport + failure certificate). By Principle 3, a bridge is accepted only if it (i) transports at least one nontrivial theorem from source to target, with a checkable proof obligation on the transported statement; (ii) generates a *testable prediction* in the target domain, to be stressed by Experimentarium; and (iii) emits a *bridge-failure certificate*: an explicit statement of where the translation breaks (which invariant is not preserved, under which perturbation the correspondence fails). A bridge that only detects semantic similarity, or that transports nothing, is rejected and stored as negative knowledge. This gate directly operationalises RQ2.

6.4 ConceptForge

ConceptForge proposes new *conceptual language* ($L3$): definitions, invariants, mathematical objects, equivalence relations, representations, categories, notions of complexity, energies, dimensions, curvatures, and convergence concepts. Its search target is a *latent structure* capable of compressing several previously separate phenomena into one.

Procedure. ConceptForge detects a set of phenomena that currently require separate explanations; searches for a common latent variable or structure; proposes an object or invariant; computes it on known cases; seeks extreme and boundary cases; formulates elementary theorems about it; and generates candidate counterexamples. The procedure is deliberately falsification-heavy: a concept is pressure-tested before it is proposed for review.

Evaluation dimensions. A candidate concept is scored (as a vector, never a scalar; see §8) on: *explanatory compression* (does it replace many conditions by one structure?); *theorem yield*; *predictive novelty* (does it predict something not used to build it?); *discriminative power* (does it separate cases that should behave differently?); *nontriviality* (is it more than a known object renamed?); *transferability*; *formalizability*; *falsifiability*; and *robustness under perturbation*. The held-out predictive test is the decisive one for RQ3: a concept fitted only to the examples that motivated it fails on held-out cases.

6.5 Experimentarium

Experimentarium is the computational laboratory of ParadigmForge. It performs finite-case enumeration, symbolic experimentation, numerical simulation, automated counterexample search, minimal-counterexample identification, invariant computation, perturbation testing, conjecture stress-testing, empirical-law discovery, and synthetic example generation.

Role, stated precisely. Experimentarium *generates evidence and falsification pressure; it does not produce proof.* Its function is to eliminate the large majority of bad ideas cheaply and to surface unexpected structure before expensive proof or review effort is spent. Every output is labelled `[EVIDENCE]` at most, and every conjecture it touches carries a record of its supporting examples, adversarial examples, known counterexamples, tested range, and unexplained failures. Certificates it produces follow the companion program's discipline: the machine-dependent step is isolated in a finite lemma with a static certificate, verified by an *independent* checker (Lucius and Minich, 2026a; Hales et al., 2017; Heule et al., 2016).

6.6 DomainFoundry

DomainFoundry is a *longitudinal evaluation* system, not a generator. It decides whether a conceptual language proposed by ConceptForge has become a productive mathematical domain ($L4$). By Principle 5, it does not mint names for alleged branches. It measures whether the proposed language: supports a natural family of objects; has meaningful internal operations; possesses useful invariants; generates multiple nontrivial theorems; resolves more than one problem family; creates natural, non-artificial internal research questions; transfers between established domains; can be used by *independent* agents; and remains productive without depending on its original proposer. A language that scores highly on all of these over a sustained horizon is a *candidate domain* (CandidateDomain); the promotion of that status is longitudinal and external, never a benchmark result (§9, Track D). This is the operational answer to RQ5.

6.7 Formalization and adversarial governance

The path from a speculative candidate to trusted knowledge is the nine-stage lifecycle of Fig. 3: (1) speculative generation; (2) computational experimentation; (3) lemma extraction; (4) informal proof construction; (5) formalization where feasible; (6) independent reconstruction; (7) adversarial review; (8) CongressBench evaluation; and (9) promotion or rejection recorded in Mirador. The authority membrane sits between stages 4 and 5: everything before it may only propose. Two invariants hold throughout. First, by Principle 2, discovery agents never have permission to promote their own claims; promotion is vested only in the independent review body, at a reviewer-independence level of at least fresh single-context review, following the Mirador separation of powers (Lucius, 2026c; Lucius and Minich, 2026b). Second, by Principle 6, a rejected proposal is written to the negative-knowledge store, not deleted (§12).

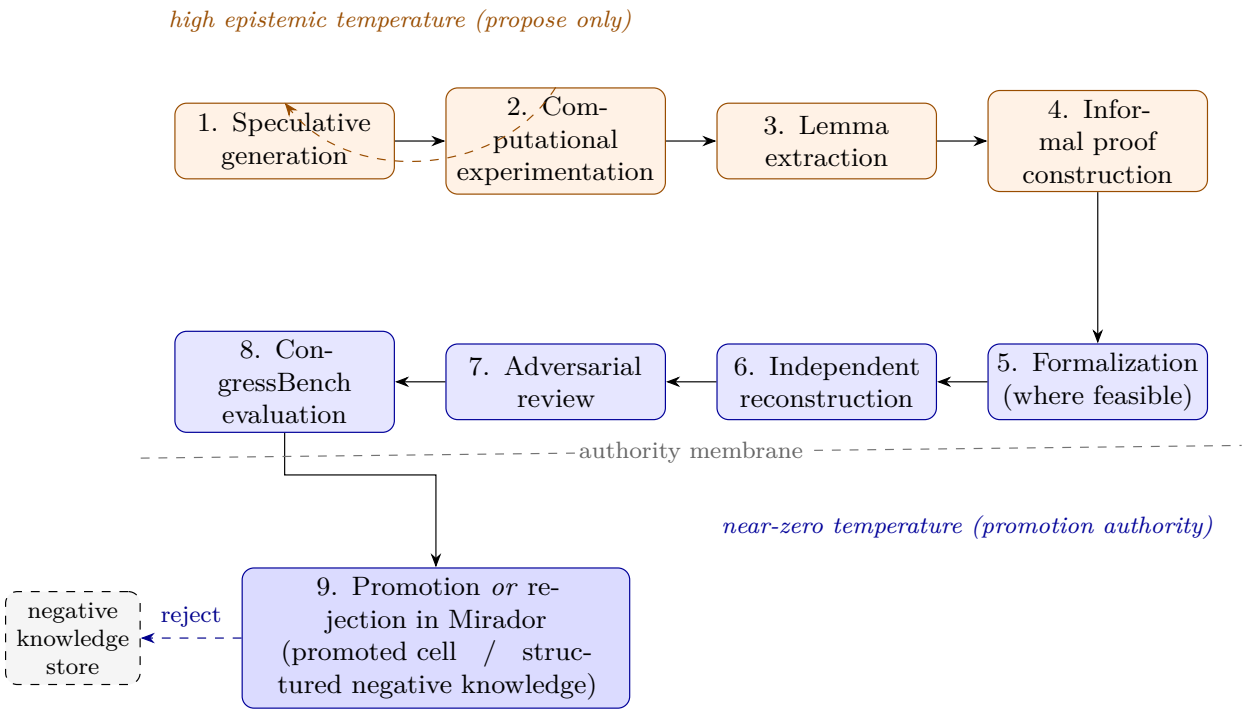

Figure 3: The discovery-to-promotion lifecycle. Stages 1–4 are high-temperature and propose-only; the authority membrane precedes formalization; stages 5–8 are near-zero-temperature and independent. A counterexample at stage 2 refutes early and cheaply; a rejection at stage 9 is stored as structured negative knowledge and fed back to the Frontier Atlas.

7 Typed Epistemic Representation

ParadigmForge does not invent a knowledge model; it extends Mirador's. The unit remains the typed, versioned Mathematical Cell, with object type orthogonal to epistemic state and identity given by the canonical hash of statement-plus-context. ParadigmForge adds node types and relation types adequate to techniques, bridges, concepts, failures, and candidate domains (Table 2). The full JSON schema is in Appendix A.

Object specification. Every ParadigmForge object carries, in addition to the Mirador core, the following fields (semi-formally; see Appendix A):

- **identity** — canonical hash of the identity-bearing statement-plus-context (a renamed object collides with its source);
- **version** — content hash plus environment (the versioned definitions it is stated against);
- **provenance** — generating operator, parent cells, Evidence-Bundle hash, model / prompt / seed, primary sources with access dates;
- **assumptions** — the identity-bearing context Γ;
- **evidence / counterevidence** — supporting and adversarial examples, tested ranges, known counterexamples;
- **epistemic_status** — one of the Mirador states (`[DRAFT]` $\to$ `[OPEN]` $\to$ `[EVIDENCE]` $\to \cdots \to$ `[PROMOTED]`, with `[REFUTED]`, `[NO_GO]`, `[SUSPECT]`);

Table 2: ParadigmForge extension of the Mirador type system. Node types (left) and relation types (right) adequate to theory formation. All are represented as typed, versioned cells / edges; object type remains orthogonal to epistemic state.

Node types	**Relation types**
MathematicalObject, Definition, Invariant, Technique, ProofSchema, Analogy, Translation, Obstruction, Counterexample, FailedApproach, ResearchBottleneck, ConceptualGap, UnexplainedPattern, CandidateDomain, ValidationArtifact	`analogous_to`, `transports_to`, `preserves`, `breaks_under`, `generalizes`, `specializes`, `dualizes`, `compresses`, `predicts`, `solves_bottleneck`, `is_obstructed_by`, `fails_because`, `depends_on`, `independently_reproduced_by`

- **novelty_status** — the novelty vector of §8, never a single scalar authority;
- **formalization_status** — {unformalized, informal-proof, formalized, certified}, with an informal–formal *correspondence* record that is never inferred from successful compilation;
- **reproducibility_status** — whether an independent agent reconstructed the object;
- **promotion_history** — the append-only log of state transitions and the review independence at each promotion.

Negative knowledge as typed objects. FailedApproach, Obstruction, and Counterexample are first-class node types with a `fails_because` relation to the target they do not solve and a `breaks_under` relation to the conditions that defeat them. A bridge-failure certificate is a ValidationArtifact attached to an Analogy or Translation. This is what makes Principle 6 mechanical rather than aspirational, and is the substrate for RQ6.

Figure 4 shows a small typed graph fragment for the illustrative campaign of §11, exhibiting the node and relation types in use.

8 Mathematical Novelty and Promotion Criteria

By Principle 7, ParadigmForge does not reduce novelty to textual distance, embedding distance, or absence from a retrieval corpus. It reports a *vector* of dimensions and gates promotion on a conjunction of criteria, not a threshold on a scalar.

8.1 Novelty dimensions

Structural novelty change in the typed structure of objects / relations, not their names.

Operational novelty a genuinely new operation (a new technique operator, a new transport), as opposed to a re-parameterisation of an existing one.

Explanatory compression reduction in the number of independent conditions or lemmas needed to state or prove a family of results.

Theorem productivity the number and depth of new theorems the object yields.

Transfer breadth the number of distinct problem families or domains the object reaches.

Counterfactual necessity whether an ablation shows the new component is required (Principle 4).

Robustness stability of the object's value under perturbation of examples and hypotheses.

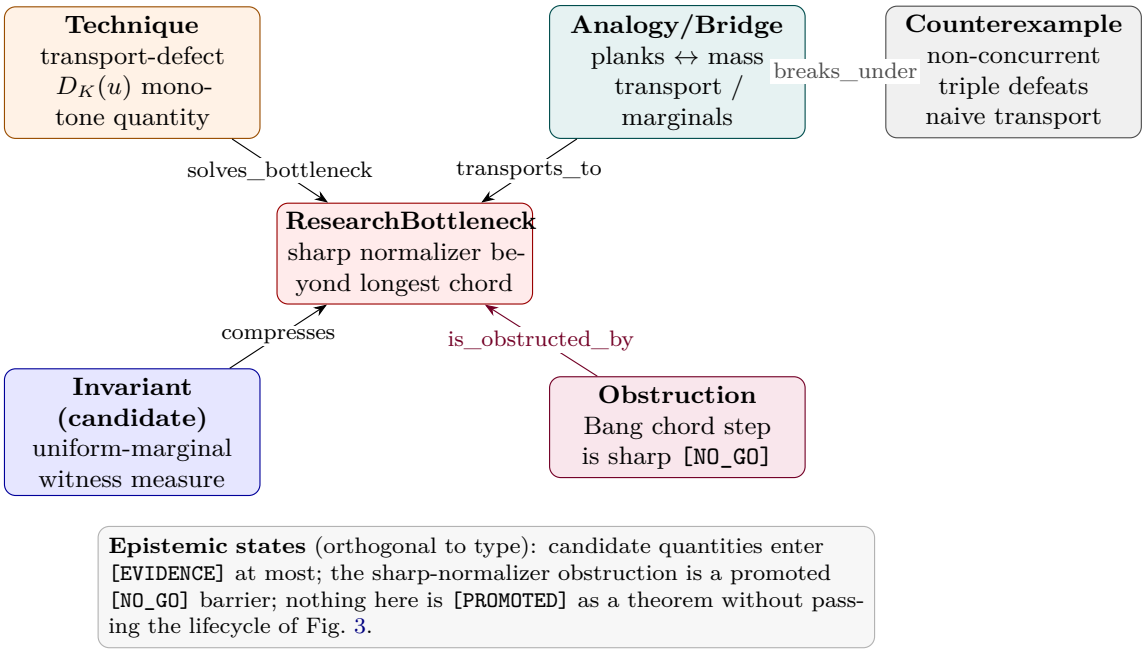

Figure 4: A typed epistemic-graph fragment for the illustrative campaign (§11). Node types and relations are drawn from Table 2; epistemic states are orthogonal to type. Candidate quantities enter `[EVIDENCE]` at most; the sharp-normalizer result is a promoted `[NO_GO]` barrier; nothing is `[PROMOTED]` as a theorem without passing the lifecycle of Fig. 3.

Independent rediscovery whether an agent with no shared context reconstructs the object.

Formal-verification coverage the fraction of the object's claims that are formalized or certified.

Proof-complexity reduction shortening of proofs that use the object.

Hypothesis-complexity reduction weakening of hypotheses that the object enables.

8.2 Scoring, and why no scalar suffices

We propose a scoring model that reports the above as a labelled vector and, *only for scheduler allocation*, a preregistered scalar aggregate. The scalar never decides promotion. The reason is structural: the dimensions are not commensurable and can move in opposite directions—a concept can have high explanatory compression yet zero independent rediscovery, or high theorem productivity on the training examples yet no held-out transfer. Collapsing them hides exactly the failure modes the system exists to catch. Promotion is therefore gated on a *conjunction* of criteria appropriate to the object's level (§12, Appendix C), not on a scalar threshold.

8.3 Anti-gaming detection

ParadigmForge includes explicit mechanisms to detect the ways an unsupervised generator can fake novelty (Table 3). Several are inherited mechanically from Mirador (content-hash identity, acyclicity-of-justification audit); the rest are dedicated checks.

Table 3: Failure modes of automated novelty and the mechanism that detects each.

Failure mode	**Detection mechanism**
Cosmetic renaming	content-hash identity collision (Mirador); flagged non-novel
Disguised restatement	semantic-equivalence check to exposed cells; requires certified link, never inferred
Trivial generalization	zero theorem yield *and* failed necessity ablation
Memorized rediscovery	memorization probe; author-perturbed variants with unpublished surface form
Benchmark contamination	held-out splits; sealed test run once; contamination controls (§9)
Circular definitions	acyclicity-of-justification audit (Mirador)
Analogy without transport	BridgeForge gate: no transported theorem ⇒ reject (Principle 3)
Concept fitted to selected examples	held-out predictive test; perturbation robustness

8.4 Promotion criteria by level

Criterion 1 (Technique, $L1$)**.** Necessity ablation passes; the technique closes a stated obligation; independent reconstruction succeeds; adversarial review finds no unresolved material objection.

Criterion 2 (Bridge, $L2$)**.** At least one nontrivial theorem is transported with a discharged proof obligation; a testable prediction is confirmed by Experimentarium; a bridge-failure certificate is present; review passes.

Criterion 3 (Concept, $L3$)**.** Explanatory compression is demonstrated; held-out predictive utility exceeds a preregistered baseline; nontriviality (not a renamed known object) is established; review passes.

Criterion 4 (Candidate domain, $L4$)**.** The longitudinal productivity criteria of §6.6 are met over a fixed horizon, including use by an independent agent and transfer to an established domain; adjudicated externally, never self-declared.

9 ParadigmForgeBench

ParadigmForgeBench is an evaluation framework—not a fixed dataset with a leaderboard—for the four levels of novelty. It has four tracks (Fig. 5). Each track specifies a task construction, a pass criterion tied to a promotion criterion, and contamination controls. **No track has been run**; task banks and sealed sets are to be built and frozen under the roadmap.

9.1 Track A: Technique reconstruction

Hide a historically important technique while exposing *only* the mathematical context that preceded it, and evaluate whether the system reconstructs a functionally equivalent method. Candidate retrospective cases—used only where the pre-discovery context can be cleanly reconstructed and the contamination risk is documented—include diagonalization, generating functions, the probabilistic method, monotonicity-quantity mechanisms in geometric flows, the polynomial method, forcing, and Fourier-analytic techniques in additive combinatorics (Cohen, 1963; Green and Tao, 2008; Guth and Katz, 2015; Hamilton, 1982; Perelman, 2002). The pass criterion is *functional*: an ablation

Track A — Technique reconstruction

Hide a historical technique; expose only the *prior* context.
Pass: reconstruct a *functionally equivalent* method (ablation-necessary), not the surface form.

Track B — Bridge discovery

Two domains; recover / construct a structural translation.
Pass: *transport a nontrivial theorem* + a testable prediction + a stated breakage point. Verbal analogy fails.

Track C — Concept formation

Families of examples, canonical abstraction withheld.
Pass: propose an invariant/concept that *compresses* the examples and *predicts held-out* behaviour.

Track D — Domain productivity

Does a candidate language yield several *independently derived* results across problem families?
Longitudinal: not scorable by a short benchmark.

All tracks enforce contamination controls, blind expert adjudication, and independent-agent replication; every claim passes the discovery-to-promotion lifecycle before it counts as "verified".

Figure 5: The four ParadigmForgeBench tracks. A–C are the primary quantitative tracks; D is longitudinal and reported qualitatively against preregistered milestones. Every track enforces contamination controls, blind expert adjudication, and independent-agent replication.

shows the reconstructed technique's novel component is necessary to close the target obligation (Principle 4); textual match to the historical statement is neither necessary nor sufficient. The first-order threat is contamination: every base technique is public and memorised by modern models, so the track requires a memorization probe run before scoring and author-perturbed variants whose surface form was never published. This track operationalises RQ1.

9.2 Track B: Bridge discovery

Provide two domains with a known (withheld) structural correspondence and a set of decoy pairs, and test whether the system recovers or constructs a meaningful translation. Evaluation requires *theorem transport*, not verbal analogy: a pass requires a typed alignment that transports at least one nontrivial theorem with a checkable obligation, a confirmed testable prediction, and a bridge-failure certificate. This track operationalises RQ2.

9.3 Track C: Concept formation

Provide families of examples with the canonical abstraction withheld and a held-out split, and test whether the system proposes an invariant or concept that compresses the visible examples and *predicts held-out behaviour* above a preregistered baseline, while surviving the overfitting probes of §8. This track operationalises RQ3.

9.4 Track D: Domain productivity

Evaluate whether a candidate conceptual language supports several *independently derived* results across multiple problem families over a fixed horizon. Full "new branch creation" cannot be evaluated by a short benchmark: it is defined by sustained autonomous productivity and must be assessed *longitudinally*, with preregistered milestones (number of independent theorems, distinct problem families, internal questions, transfer, and use by a non-proposer agent), never as a single success rate. This track operationalises RQ5.

10 Experimental Methodology

We describe a credible empirical program. **It is proposed, not executed**; the full phase schedule, blocking decisions, and preregistration ritual are in the accompanying roadmap. The purpose here is to make the architecture falsifiable.

Design. Retrospective rediscovery experiments (Track A); held-out mathematical domains (Tracks B, C); contamination controls (memorization probes, perturbed variants, sealed test run once); blind expert evaluation; independent-agent replication; adversarial counterexample search; formal verification where possible; and comparison against strong baselines: a direct language-model prover/reasoner, a retrieval-only system, unguided multi-agent debate, and TheoryForge without the ParadigmForge layer. Compute and inference budgets are reported for every arm.

Ablations. The program removes, one at a time: the Frontier Atlas (undirected exploration); negative knowledge; BridgeForge; Experimentarium (no computational falsification pressure); independent review (self-review only); Mirador's typed representation (prose cells); formal verification; and the domain-transfer requirement. The central governance test (RQ4) is whether removing independent review *raises* the FTPR and the cosmetic-novelty escape rate; the negative-knowledge test (RQ6) is whether removing the failure store increases repeated failure and time-to-first-useful-candidate.

Statistics. Comparisons are paired on shared items (clustered by base problem/domain), with the design effect reported; effect sizes with intervals, not only p-values; multiplicity controlled; the sealed test executed once with no optional stopping. Every table regenerates deterministically from raw artifacts.

Placeholders, not results. Table 4 shows the frozen shape of the primary results reporting. **Its cells are intentionally empty**; they are filled only after preregistration freeze and the runs specified in the roadmap. Nothing in this paper is a finding.

Table 4: **[PENDING — no experiment run]** Frozen shape of the primary ParadigmForgeBench results. Cells are filled only after preregistration freeze; this is a reporting template, not data.

Track	Arm	Pass rate	Necessity-ablation	Transfer	Compute (rel.)
A (technique)	B-LLM / B-RET / B-DEBATE / B-TF / PF	*[pending]*	*[pending]*	—	*[pending]*
B (bridge)	B-LLM / B-RET / B-DEBATE / B-TF / PF	*[pending]*	—	*[pending]*	*[pending]*
C (concept)	B-LLM / B-RET / B-DEBATE / B-TF / PF	*[pending]*	—	*[pending]*	*[pending]*
D (domain)	PF (longitudinal)	*[longitudinal — milestone trajectory, not a scalar]*			

11 Illustrative Research Campaign

We give a technically serious illustration of the workflow on an open but tractable problem already documented in the Bourbaki corpus: *Bang's affine plank problem* (Bang, 1951; Ball, 1991; Gardner, 1988; Lucius, 2026a). The purpose is to demonstrate the components in motion, *not* to solve the problem. It is academically legitimate—and here more informative—for the illustration to end in a structured, well-explained non-result.

The problem. A *plank* of width w is the region between two parallel hyperplanes at distance w. For a convex body K and a plank P with normal u, the relative width is $\mathrm{rw}_K(P) = \mathrm{width}(P)/\mathrm{width}_K(u)$. Bang's affine plank conjecture (1951) asserts that if K is covered by planks $P_1, \ldots, P_n$ then $\sum_i \mathrm{rw}_K(P_i) \geq 1$. Ball proved the centrally symmetric case (Ball, 1991); the general (affine) case is open, already in the plane and already for three planks, and Gardner posed the associated relative-width witness-measure existence question (Gardner, 1988).

1. Frontier Atlas. Ingesting the plank literature and the documented campaign (Lucius, 2026a), the Atlas assembles known results (Ball's symmetric theorem; the transport-defect framework $D_K(u)$; a median theorem with rigidity on the triangle; a three-direction characterization settling Gardner's question for three directions), techniques and their regimes, and barriers.

2. Bottleneck. The Atlas localises a precise ResearchBottleneck: existing arguments control the covering constant only up to the *one-plank-per-direction* restriction (C^{111}), and the step that would extend a bound to the unrestricted constant C passes through a normalizer argument that is *sharp*—Bang's chord-lemma step cannot reach any normalizer beyond the longest chord. The bottleneck cell names exactly this obligation: transport a bound from C^{111} to C without the chord-lemma normalizer. (This is also the site of the campaign's documented flagship defect: a value proved for C^{111} but stated for C, where $C \leq C^{111}$ makes the two non-interchangeable (Lucius, 2026c).)

3. Candidate technique (TechniqueForge). A candidate technique proposes to *quantitatively strengthen* the transport-defect argument by coupling energy-type estimates to a concentration-of-measure control on the marginal defect, aiming for an almost-monotone quantity that survives the passage from C^{111} to C. The candidate is emitted at `[EVIDENCE]` with a necessity-ablation obligation: remove the concentration control and check whether the transport step still closes.

4. Candidate bridge (BridgeForge). A candidate Analogy/Translation aligns plank coverings with a mass-transport / marginal-coupling formulation: planks and directions map to marginals, relative width to a transport cost, and the covering inequality to a marginal-defect bound. The bridge is required to *transport* a nontrivial statement (e.g. the symmetric case's marginal

identity) and to emit a failure certificate. Experimentarium immediately supplies a candidate COUNTEREXAMPLE: a non-concurrent tilted triple defeats the naive transport, so the bridge is annotated `breaks_under` "non-concurrent triples" rather than promoted.

5. Candidate invariant (ConceptForge). A candidate INVARIANT—a uniform-marginal witness measure whose existence is equivalent to a concurrence condition on mid-level lines—is proposed as the object that `compresses` the three-direction phenomena. It is computed on known cases and scored on explanatory compression and held-out prediction; on the triangle it recovers the documented three-direction characterization but does *not*, in this illustration, extend to general triples.

6. Computational tests (Experimentarium). Finite enumeration and exact-rational certificate search stress the candidates: the one-plank-per-direction constant at a specific tilted triple is confirmed by an independent checker over a large exact certificate (Lucius, 2026a), while the unrestricted constant remains open; the candidate almost-monotone quantity fails to remain monotone on a perturbed family, producing falsification pressure rather than support.

7. Lemma extraction. The MINIMALMISSINGLEMMA is extracted: a "thin-plank" lemma that would lift the finite branch-and-bound to the full cyclic family. It is emitted `[OPEN]` with a `solves_bottleneck` edge—the precise statement whose absence blocks the campaign.

8. Adversarial objections. A fresh-context review (CongressBench-conforming) raises: (i) the transported statement in step 4 holds only under concurrence (scope objection); (ii) the almost-monotone quantity is not monotone off the perturbed family (necessity-ablation failure); (iii) the C^{111}-vs-C scope must not be elided (the documented flagship defect). Each objection is a structured veto with a discharge condition.

9. Outcome. The honest outcome of this illustration is a *structured non-result*: the sharp-normalizer obstruction is promoted as a `[NO_GO]` *barrier* (a first-class asset that prunes an entire method family); the candidate technique and bridge are `[REFUTED]` on the perturbed family and stored as FAILEDAPPROACH with `fails_because` explanations; the uniform-marginal invariant is retained at `[EVIDENCE]` for three directions with an explicit non-extension note; and the thin-plank lemma is recorded as the ranked next target. No theorem is promoted, no problem is claimed solved, and the negative knowledge produced is precisely the material that sharpens the Frontier Atlas for the next cycle (RQ6). Figure 4 shows the resulting typed graph fragment.

12 Governance, Failure and Negative Knowledge

Governance is what separates ParadigmForge from an ideation loop. Three mechanisms carry it.

Separation of powers. By Principle 2, generation and promotion are distinct authorities and never vested in the same agent. A discovery agent may propose an object at `[DRAFT]`/`[EVIDENCE]`; only the independent review body may promote, and only through the Mirador transition table, which mechanically forbids self-promotion and requires a fresh-context reviewer at independence level ≥ 3 with no unresolved veto (Lucius, 2026c). In the implementation this is a database-permission fact, not a policy request: the generating role holds no write access to the promoted corpus (Lucius and Minich, 2026a).

Adversarial review. Promotion runs through a CongressBench-conforming panel (Lucius and Minich, 2026b) whose lenses are tuned to ParadigmForge's failure modes: a necessity-ablation auditor (was the new component required?); a transport auditor (does the bridge carry a theorem?); a scope/assumption auditor (the C^{111}-vs-C lens); a novelty-and-priority auditor (is this a renamed known object?); and an independent-reconstruction auditor. Whether fresh-context review lowers the FTPR relative to shared-context review is the empirical wager the companion benchmark is built to settle; ParadigmForge inherits it as a hypothesis (RQ4), not a fact.

Negative knowledge as a first-class output. By Principle 6, rejection is recorded, not discarded. A refuted technique becomes a FailedApproach with a `fails_because` edge; a defeated bridge yields a bridge-failure ValidationArtifact; a proved impossibility becomes a Obstruction in state `[NO_GO]`, valued at parity with a positive result because it prunes infinite search. The Frontier Atlas consumes these directly: a family of approaches that fail for a common reason collapses to one high-value cell, so the system does not re-attempt known-dead routes. This is the mechanism behind RQ6, and it is the counterpart, at the level of theory formation, of the invalidation propagation that Mirador provides at the level of claims.

13 Limitations and Threats to Validity

We state the limitations bluntly, because the paper's credibility depends on not overclaiming.

Maturity. ParadigmForge is *specified, not built.* The substrates it depends on (Mirador, ProofContext) are implemented prototypes; the six ParadigmForge components, the type-system extension, and ParadigmForgeBench are designs. Two dependencies are known blockers: ProofContext currently exposes neither cross-domain analogy retrieval nor a collision-search client, which are prerequisites for BridgeForge and for the first novelty-screening stage (Lucius and Minich, 2026c). No ParadigmForge experiment has been run.

No results. This paper contains no accuracy, success rate, expert assessment, quotation, table of measurements, or mathematical discovery. Every results table is a labelled placeholder. Claims are architectural and methodological only.

Contamination. The retrospective tracks rely on techniques and problems that are public and memorised by modern models. Contamination is mitigated (memorization probes, perturbed variants, functional rather than textual scoring, sealed tests) but cannot be eliminated; Track A results, once produced, must be read with that caveat, and rediscovery must never be reported as novelty.

Evaluation of the hardest levels. $L4$ (domain genesis) is not evaluable by any short benchmark and is deliberately left to longitudinal, external assessment. Even Tracks A–C, being built by the same program that designs the mechanisms they reward, carry a construct-validity risk that only a held-out or third-party benchmark can resolve.

The generator is a bound. ParadigmForge does not manufacture insight. A campaign is bounded by the best technique, bridge, or concept its generators can propose, and by representation bottlenecks, formalization cost, retrieval error, and model homogenisation. Governance can prevent false promotion; it cannot create tractability.

Anthropomorphism. We avoid claims of "mathematical creativity" or "understanding". ParadigmForge proposes candidate objects and governs their validation; whether the resulting process resembles human discovery is out of scope.

14 Relationship with the Clay Millennium Problems

We discuss the Millennium Prize Problems only as a long-term motivating horizon, and we make no claim that ParadigmForge can solve one. The reason to mention them is that they are precisely the class of problems for which proving-in-a-fixed-language is plausibly insufficient. Several may require *new techniques*, *unexpected bridges*, *new conceptual languages*, or frameworks not yet available—the very objects ParadigmForge is designed to propose and govern. Historically, decisive progress on hard problems has come through exactly such representational change: forcing for the continuum hypothesis (Cohen, 1963), Ricci-flow monotonicity for the geometrisation and Poincaré programs (Hamilton, 1982; Perelman, 2002), the polynomial method for incidence problems (Guth and Katz, 2015), and Fourier-analytic transference in additive combinatorics (Green and Tao, 2008).

Two distinctions matter. First, olympiad problem solving is not research-level theory formation: the former discharges statements in a settled language under time pressure (Trinh et al., 2024; Chervonyi et al., 2025); the latter changes the language—closer in spirit to research-level benchmarks of unpublished problems (Abouzaid et al., 2026)—and its success predicate is external and slow. Second, a Millennium "solution" cannot be defined by an internal agent verdict: it is closed only by the external mathematical process (journal acceptance, community validation), which no system can short-circuit (Lucius and Minich, 2026a). ParadigmForge is therefore positioned not as a solver but as a test of whether AI can participate in *constructing the conceptual machinery* that frontier problems may require—and, just as importantly, of whether that participation can be governed so that its speculative output does not contaminate trusted knowledge.

15 Research Roadmap

The execution plan is preregistered in the accompanying roadmap document; we summarise its spine. The program proceeds in phases: extend the Mirador schema and validators (P0); build the Frontier Atlas over a frozen corpus (P1); implement Experimentarium with exact-arithmetic re-checking (P2); implement the TechniqueForge / BridgeForge / ConceptForge operators and the necessity-ablation harness (P3); *freeze* the preregistration—protocols, corpora manifests, ablation matrix, novelty-rubric weights, split hashes—as a hash-pinned event (P4); run Tracks A–C and the governance ablations (P5–P7); pilot the longitudinal Track D under external expert governance (P8); and execute the sealed test once, reporting every number regenerated from raw artifacts, including null and negative results (P9). Blocking decisions—compute budget, model families for the independence requirement, retrospective-corpus selection and licensing, the external expert panel, the held-out concept/bridge targets, and the ProofContext analogy/collision extensions—are enumerated there and must be signed off before P4.

16 Conclusion

ParadigmForge extends the Bourbaki Engine from the governance of claim promotion to the governance of theory formation. Its wager is that progress toward autonomous mathematical discovery requires systems that can change the representational and conceptual space in which problems are posed—not only prove statements inside a fixed language—and that such change is safe

only under an explicit separation of temperatures: permissive, diverse, contradictory exploration on one side; conservative, auditable, independent promotion on the other; every rejection retained as knowledge. We have specified an architecture for this (Frontier Atlas, TechniqueForge, BridgeForge, ConceptForge, Experimentarium, DomainFoundry), a typed representation for techniques, bridges, concepts, and failures, a multidimensional novelty model with explicit anti-gaming controls, and ParadigmForgeBench, a four-track evaluation framework that measures productivity rather than textual novelty. We have been explicit that the ParadigmForge layer is not yet implemented and that no experiment has been run; the paper's contributions are architectural and methodological, and its results tables are placeholders.

The defensible claim is narrow and, we think, correct: mathematical AI will not reach frontier discovery merely by proving more statements inside fixed languages. It must also learn to propose, test, reject, and *govern* changes to the conceptual language of mathematics itself. ParadigmForge is a design for doing so without letting speculation masquerade as knowledge.

Reproducibility and Availability

The LaTeX source, figure sources, a README, and the full experimental roadmap accompany this paper. No datasets or result artifacts are released, because none exist yet: this is a pre-execution architecture paper. The companion Bourbaki prototypes are separately available (Lucius, 2026c,d).

References

Mohammed Abouzaid, Andrew J. Blumberg, Martin Hairer, Joe Kileel, Tamara G. Kolda, Paul D. Nelson, Daniel Spielman, Nikhil Srivastava, Rachel Ward, Shmuel Weinberger, and Lauren Williams. First proof. *arXiv preprint arXiv:2602.05192*, 2026. URL https://arxiv.org/abs/2602.05192.

Keith Ball. The plank problem for symmetric bodies. *Inventiones Mathematicae*, 104:535–543, 1991. doi: 10.1007/BF01245089.

Thoger Bang. A solution of the "plank problem". *Proceedings of the American Mathematical Society*, 2(6):990–993, 1951. doi: 10.2307/2031721.

Alan Bundy. The use of explicit plans to guide inductive proofs. In *Automated Deduction (CADE-9)*, volume 310 of *Lecture Notes in Computer Science*, pages 111–120. Springer, 1988.

Ahmet Çelik, Karl Palmskog, and Milos Gligoric. iCoq: Regression proof selection for large-scale verification projects. In *Proc. 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2017.

Yuri Chervonyi, Trieu H. Trinh, Miroslav Olšák, Xiaomeng Yang, Hoang Nguyen, Marcelo Menegali, Junehyuk Jung, Junsu Kim, Vikas Verma, Quoc V. Le, and Thang Luong. Gold-medalist performance in solving olympiad geometry with AlphaGeometry2. *arXiv preprint arXiv:2502.03544*, 2025. URL https://arxiv.org/abs/2502.03544.

François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019. URL https://arxiv.org/abs/1911.01547.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John

Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL https://arxiv.org/abs/2110.14168.

Paul J. Cohen. The independence of the continuum hypothesis. *Proceedings of the National Academy of Sciences*, 50(6):1143–1148, 1963. doi: 10.1073/pnas.50.6.1143.

Simon Colton. *Automated Theory Formation in Pure Mathematics*. Distinguished Dissertations. Springer, London, 2002. doi: 10.1007/978-1-4471-0147-5.

Antoine Cully, Jeff Clune, Danesh Tarapore, and Jean-Baptiste Mouret. Robots that can adapt like animals. *Nature*, 521(7553):503–507, 2015. doi: 10.1038/nature14422.

Alex Davies, Petar Veličković, Lars Buesing, Sam Blackwell, Daniel Zheng, Nenad Tomašev, Richard Tanburn, Peter Battaglia, Charles Blundell, András Juhász, Marc Lackenby, Geordie Williamson, Demis Hassabis, and Pushmeet Kohli. Advancing mathematics by guiding human intuition with AI. *Nature*, 600(7887):70–74, 2021. doi: 10.1038/s41586-021-04086-x.

Johan de Kleer. An assumption-based TMS. *Artificial Intelligence*, 28(2):127–162, 1986. doi: 10.1016/0004-3702(86)90080-9.

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021. doi: 10.1007/978-3-030-79876-5_37.

Jon Doyle. A truth maintenance system. *Artificial Intelligence*, 12(3):231–272, 1979. doi: 10.1016/0004-3702(79)90008-0.

Richard J. Gardner. Relative width measures and the plank problem. *Pacific Journal of Mathematics*, 135(2):299–312, 1988. doi: 10.2140/pjm.1988.135.299.

Dedre Gentner. Structure-mapping: A theoretical framework for analogy. *Cognitive Science*, 7(2):155–170, 1983. doi: 10.1207/s15516709cog0702_3.

Elliot Glazer, Ege Erdil, Tamay Besiroglu, Diego Chicharro, Evan Chen, Alex Gunning, Caroline Falkman Olsson, Jean-Stanislas Denain, Anson Ho, Emily de Oliveira Santos, et al. FrontierMath: A benchmark for evaluating advanced mathematical reasoning in AI. *arXiv preprint arXiv:2411.04872*, 2024. URL https://arxiv.org/abs/2411.04872.

Georges Gonthier. Formal proof—the four-color theorem. *Notices of the American Mathematical Society*, 55(11):1382–1393, 2008.

Google DeepMind. AI achieves silver-medal standard solving international mathematical olympiad problems. Google DeepMind Blog, July 2024. URL https://deepmind.google/blog/ai-solves-imo-problems-at-silver-medal-level/. Published 25 July 2024.

Ben Green and Terence Tao. The primes contain arbitrarily long arithmetic progressions. *Annals of Mathematics*, 167(2):481–547, 2008. doi: 10.4007/annals.2008.167.481.

Larry Guth and Nets Hawk Katz. On the Erdős distinct distances problem in the plane. *Annals of Mathematics*, 181(1):155–190, 2015. doi: 10.4007/annals.2015.181.1.2.

Thomas Hales, Mark Adams, Gertrud Bauer, Tat Dat Dang, John Harrison, Hoang Le Truong, Cezary Kaliszyk, Victor Magron, Sean McLaughlin, Tat Thang Nguyen, et al. A formal proof of the Kepler conjecture. *Forum of Mathematics, Pi*, 5:e2, 2017. doi: 10.1017/fmp.2017.1.

Richard S. Hamilton. Three-manifolds with positive Ricci curvature. *Journal of Differential Geometry*, 17(2):255–306, 1982. doi: 10.4310/jdg/1214436922.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2021. URL https://arxiv.org/abs/2103.03874.

Marijn J. H. Heule, Oliver Kullmann, and Victor W. Marek. Solving and verifying the Boolean Pythagorean triples problem via cube-and-conquer. In *Theory and Applications of Satisfiability Testing (SAT 2016)*, volume 9710 of *Lecture Notes in Computer Science*, pages 228–245. Springer, 2016. doi: 10.1007/978-3-319-40970-2_15.

Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2210.12283.

Tobias Kuhn, Christine Chichester, Michael Krauthammer, Núria Queralt-Rosinach, Ruben Verborgh, George Giannakopoulos, Axel-Cyrille Ngonga Ngomo, Raffaele Viglianti, and Michel Dumontier. Decentralized provenance-aware publishing with nanopublications. *PeerJ Computer Science*, 2:e78, 2016. doi: 10.7717/peerj-cs.78.

Imre Lakatos. *Proofs and Refutations: The Logic of Mathematical Discovery*. Cambridge University Press, 1976.

Guillaume Lample, Marie-Anne Lachaux, Thibaut Lavril, Xavier Martinet, Amaury Hayat, Gabriel Ebner, Aurélien Rodriguez, and Timothée Lacroix. HyperTree proof search for neural theorem proving. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2205.11491.

Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. RFC 6962, IETF, 2013.

Joel Lehman and Kenneth O. Stanley. Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary Computation*, 19(2):189–223, 2011. doi: 10.1162/EVCO_a_00025.

Douglas B. Lenat. *AM: An Artificial Intelligence Approach to Discovery in Mathematics as Heuristic Search*. PhD thesis, Stanford University, 1976. Stanford AI Lab Memo AIM-286, STAN-CS-76-570.

Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving quantitative reasoning problems with language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2206.14858.

Maximiliano Lucius. Transport and tiling bounds for the affine plank problem on the triangle. Manuscript, 35 pp.; MSC 52C17; repository maximilianolucius/bang-plank-affine-triangle, 2026a.

Maximiliano Lucius. MathIngestBench: Measuring faithful domain ingestion from mathematical textbooks. Zenodo, 2026b. URL https://doi.org/10.5281/zenodo.21329350.

Maximiliano Lucius. MIRADOR: A typed, versioned intermediate representation for auditable mathematical knowledge and discovery. Zenodo, 2026c. URL https://doi.org/10.5281/zenodo.21324524.

Maximiliano Lucius. ProofContext: Dependency-aware retrieval of verifiable context for mathematical reasoning. Zenodo, 2026d. URL https://doi.org/10.5281/zenodo.21325289.

Maximiliano Lucius and José Minich. The Bourbaki engine: A governed discovery architecture for long-horizon autonomous mathematics. Zenodo, 2026a. URL https://doi.org/10.5281/zenodo.21328015.

Maximiliano Lucius and José Minich. CongressBench: Measuring false theorem promotion in agentic mathematics. Zenodo, 2026b. URL https://doi.org/10.5281/zenodo.21328878.

Maximiliano Lucius and José Minich. TheoryForge: Operator-guided autonomous theory formation in mathematics. Zenodo, 2026c. URL https://doi.org/10.5281/zenodo.21329229.

William McCune. Solution of the Robbins problem. *Journal of Automated Reasoning*, 19(3):263–276, 1997. doi: 10.1023/A:1005843212881.

Luc Moreau and Paolo Missier. PROV-DM: The PROV data model. W3c recommendation, World Wide Web Consortium (W3C), 2013. 30 April 2013.

Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015. URL https://arxiv.org/abs/1504.04909.

Allen Newell, J. C. Shaw, and Herbert A. Simon. Empirical explorations of the logic theory machine: A case study in heuristic. In *Proc. Western Joint Computer Conference*, pages 218–230, 1957.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025. URL https://arxiv.org/abs/2506.13131.

Grisha Perelman. The entropy formula for the Ricci flow and its geometric applications. *arXiv preprint arXiv:math/0211159*, 2002. URL https://arxiv.org/abs/math/0211159.

Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020. URL https://arxiv.org/abs/2009.03393.

George Pólya. *How to Solve It*. Princeton University Press, 1945.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. doi: 10.1038/s41586-023-06924-6.

The mathlib Community. The Lean mathematical library. In *Proc. 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 367–381, 2020. doi: 10.1145/3372885.3373824.

Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024. doi: 10.1038/s41586-023-06747-5.

Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2205.12615.

Huajian Xin, Daya Guo, Zhihong Shao, Zhizhou Ren, Qihao Zhu, Bo Liu, Chong Ruan, Wenda Li, and Xiaodan Liang. DeepSeek-Prover: Advancing theorem proving in LLMs through large-scale synthetic data. *arXiv preprint arXiv:2405.14333*, 2024a. URL https://arxiv.org/abs/2405.14333.

Huajian Xin, Z. Z. Ren, Junxiao Song, Zhihong Shao, Wanjia Zhao, Haocheng Wang, Bo Liu, Liyue Zhang, Xuan Lu, Qiushi Du, Wenjun Gao, Qihao Zhu, Dejian Yang, Zhibin Gou, Z. F. Wu, Fuli Luo, and Chong Ruan. DeepSeek-Prover-V1.5: Harnessing proof assistant feedback for reinforcement learning and monte-carlo tree search. *arXiv preprint arXiv:2408.08152*, 2024b. URL https://arxiv.org/abs/2408.08152.

Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023. URL https://arxiv.org/abs/2306.15626.

Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. MiniF2F: a cross-system benchmark for formal olympiad-level mathematics. In *International Conference on Learning Representations (ICLR)*, 2022. URL https://arxiv.org/abs/2109.00110.

A Object Schemas

A semi-formal specification of the ParadigmForge extension cell (JSON-like; the executable schema targets Mirador's `mirador-cell/1.0` and adds the fields below). Object type is orthogonal to epistemic state; identity is the canonical hash of the identity-bearing statement and context.

```
ParadigmForgeCell {
  id                    : content-hash(statement + identity-bearing context)   // identity
  version               : hash(content_hash + environment)                     // version axis
  object_type           : one of { MathematicalObject, Definition, Invariant,
                                   Technique, ProofSchema, Analogy, Translation,
                                   Obstruction, Counterexample, FailedApproach,
                                   ResearchBottleneck, ConceptualGap,
                                   UnexplainedPattern, CandidateDomain,
                                   ValidationArtifact }
  epistemic_status      : one of { DRAFT, OPEN, EVIDENCE, CANDIDATE, FORMALIZED,
                                   AUDITED, PROMOTED, REFUTED, NO_GO, SUSPECT,
                                   RETIRED }                    // orthogonal to type
  assumptions           : context Gamma { variables, assumptions, exclusions,
                                          regularity, scope }    // identity-bearing
  statement_ir          : typed intermediate representation      // identity-bearing
  provenance            : { operator, parents[], evidence_bundle_hash,
                            model, prompt, seed, primary_sources[] with access_dates }
  evidence              : { supporting_examples[], tested_range }
  counterevidence       : { adversarial_examples[], known_counterexamples[],
                            unexplained_failures[] }
  novelty_status        : NoveltyVector {                        // never a scalar authority
                            structural, operational, explanatory_compression,
                            theorem_productivity, transfer_breadth,
```

```
                          counterfactual_necessity, robustness,
                          independent_rediscovery, formal_coverage,
                          proof_complexity_reduction, hypothesis_complexity_reduction }
  formalization_status  : { unformalized | informal_proof | formalized | certified }
  correspondence_status : { asserted | reviewed | unchecked }   // never inferred from compile
  reproducibility_status: { none | independently_reproduced_by: agent_id }
  edges                 : typed[ analogous_to, transports_to, preserves, breaks_under,
                                 generalizes, specializes, dualizes, compresses,
                                 predicts, solves_bottleneck, is_obstructed_by,
                                 fails_because, depends_on, independently_reproduced_by ]
  promotion_history     : append-only[ {state, actor, review_independence, ts} ]
}
```

B Agent Roles

ParadigmForge inherits the Bourbaki role vocabulary and adds theory-formation roles. Generation and promotion authorities are disjoint (Principle 2).

- **Cartographer** — builds and maintains the Frontier Atlas; owns ResearchBottleneck cells.
- **Technician** — runs TechniqueForge operators; emits Technique candidates ($\leq$ `[EVIDENCE]`).
- **Bridge-builder** — runs BridgeForge; emits Analogy/Translation with transport obligations.
- **Conceptor** — runs ConceptForge; emits Definition/Invariant candidates.
- **Experimenter** — runs Experimentarium; produces `[EVIDENCE]` and counterexamples, never proof.
- **Formalizer** — attempts formalization / certificate extraction.
- **Reconstructor** — an independent agent that re-derives a candidate from scratch (fresh context).
- **Congress (Referees)** — the only promotion authority; runs the adversarial lenses of §12.
- **Archivist** — writes rejections to the negative-knowledge store; owns the event log.

No discovery role holds write access to the promoted corpus.

C Promotion Gates

Promotion is a conjunction of gates, by level. Each gate is a checkable predicate; failing any gate returns the object to the speculative zone (or to negative knowledge if refuted).

Level	Conjunctive promotion gates
$L1$ Technique	necessity-ablation pass $\wedge$ obligation closed $\wedge$ independent reconstruction $\wedge$ no unresolved veto
$L2$ Bridge	≥ 1 theorem transported (obligation discharged) $\wedge$ confirmed prediction $\wedge$ failure certificate present $\wedge$ review pass
$L3$ Concept	explanatory compression $\wedge$ held-out prediction $>$ baseline $\wedge$ nontriviality (no identity collision) $\wedge$ review pass
$L4$ Domain	longitudinal productivity criteria (§6.6) over horizon $\wedge$ independent-agent use $\wedge$ external adjudication

D Evaluation Rubrics

Partial-credit rubrics are reported per dimension, never fused into one number. Sketch:

- **Technique (Track A)** — {no candidate; candidate but ablation-unnecessary; necessary but obligation not closed; obligation closed; independently reconstructed}.
- **Bridge (Track B)** — {verbal analogy only; typed alignment, no transport; transports a trivial statement; transports a nontrivial theorem; transport + confirmed prediction + failure certificate}.
- **Concept (Track C)** — {renamed known object; compresses training only; compresses + weak held-out; compresses + strong held-out; + transfer to a second family}.
- **Domain (Track D)** — milestone trajectory: objects family; internal operations; invariants; multi-theorem; multi-family; internal questions; transfer; non-proposer use.

E Example Research Artifacts

Two illustrative negative-knowledge artifacts (schematic; not results).

```
FailedApproach {
  object_type      : FailedApproach
  epistemic_status : REFUTED
  target           : bottleneck: transport bound from C^{111} to C without chord-lemma normalizer
  approach         : almost-monotone quantity via energy + concentration control
  fails_because    : quantity not monotone on the perturbed cyclic family (Experimentarium)
  breaks_under     : { perturbation: off-cyclic tilt }
  reusable         : yes  // prunes the "energy+concentration" family for this bottleneck
}

BridgeFailureCertificate {
  object_type      : ValidationArtifact
  attached_to      : Analogy(planks <-> mass-transport marginals)
  transported      : symmetric-case marginal identity   // holds
  breaks_under      : non-concurrent tilted triples      // transport fails
  prediction_status: refuted by counterexample (Experimentarium)
}
```

F Pseudocode: Discovery and Governance Loops

The discovery loop (high temperature, propose-only) and the governance loop (near-zero temperature, promotion authority) are disjoint in authority.

```
# Discovery loop (speculative zone; emits cells <= EVIDENCE only)
def discovery_cycle(target):
    atlas       = FrontierAtlas.build(target, corpus, negative_knowledge)
    bottleneck  = atlas.select_bottleneck()          # ResearchBottleneck
    bundle      = ProofContext.query(bottleneck).bundle
    candidates  = []
    candidates += TechniqueForge.generate(bottleneck, bundle)   # L1
    candidates += BridgeForge.generate(bottleneck, bundle)      # L2
    candidates += ConceptForge.generate(bottleneck, bundle)     # L3
    for c in candidates:
        c = Experimentarium.stress(c)               # evidence + counterexamples, not proof
```

```
        if c.refuted:
            NegativeKnowledge.store(c)              # FailedApproach / Obstruction
            continue
        if not necessity_ablation_passes(c):        # Principle: necessity over decoration
            NegativeKnowledge.store(c); continue
        Mirador.write(c, state=EVIDENCE)            # NO promotion authority here
        ReviewQueue.submit(c)
    return

# Governance loop (trust zone; sole promotion authority)
def governance_cycle():
    for c in ReviewQueue:
        assert c.generator != current_reviewer      # separation of powers
        c = Formalizer.try(c)                        # where feasible
        c = Reconstructor.independent(c)             # fresh context
        verdict = Congress.review(c, lenses=[necessity, transport, scope,
                                             novelty_priority, reconstruction])
        if verdict.accept and gates_pass(c, c.level):
            Mirador.promote(c, independence=verdict.level)   # only path to PROMOTED
        else:
            NegativeKnowledge.store(c, reason=verdict.vetoes)  # failure is knowledge
    return
```